14 | 10 | 2026

Homework Read and summarize the following slides in your notebook

Slide 1 05.01 => Modeling Practices

Conceptual modeling involves capturing ideas and concepts in a form that facilitates such as reasoning, analysis, and design. The primary purpose of this practice is to gain a better understanding of a real-world system - visualizing it as it currently exists or as we intend for it to become as well as to document key decisions and communicate efficiently. This type of modeling can be applied at any stage of the software development process, whether from the perspective of analysis, design, specification, implementation or deployment. To achieve this UML is used it provides various tools such as use case diagrams, class diagrams, interaction diagrams and stak diagrams to describe the objects within a system and their behaviors.

Object Oriented Best Practices: To build a robust software system, it is advisable to organize in into loosely coupled layers, where each layer provides a well defined abstraction to the layers above it. Regarding code reuse, aggregation should be preferred over inheritance, as aggregation relationships can be modified at runtime, offering significantly greater flexibility. Another approach to managing coupling and eliminating unnecessary dependencies is to program to an interface or abstraction rather than

Escaneado con CamScanner

to concrete implementations.
Furthermore, software readability and maintainability improve significantly whe choosing identifiers that make sense within their context. For example, class names should be nouns written in "Pascal Casing", variables should reflect their purpose using "lowerCamelCasing", and methods should be verbs indicating an action. To ensure quality, testing should be conducted using executable unit test cases; this allows for the detection of design errors, repeatable verification of code correctness, the enabling of regression testing, and the documentation of system usage. Finally making class variables: private is highlighted as an essential practice, as leaving them publicly accesible typically violates the encapsulation principle of the object oriented paradigm.

Slide 1.05.07 => Introduction To UML

UML is a set of object oriented modeling languages characterized by being expressive, semi-formal, independt of the development process, and customizable. It is built from basic elements that can be structural, behavioral, organizational, or annotational. These elements are combined to form diagrams and the diagrams taken together constitute complete models.

Modeling Perspectives and Views: A system is described through "views" sets of diagrams that present the system from a especific perspective, such as the logical view or the implementation view. In

software engineering, there are three fundamental perspectives: analysis, which focuses on understanding real world objects and the problem at hand specification, which is oriented toward defining the system's boundaries to address problems and implementation which centers on the construction and software components of the system.

Types of Diagrams: Diagrams fall into two broad categories. Structural diagrams illustrate the static parts of the system and include class, object, component, and deployment diagrams. On the other hand, behavioral diagrams show the system's dynamics and include use case, sequence, collaboration, state machine, and activity diagrams.

Modeling Best Practices: The primary goal of creating diagrams is not to produce attractive images, but rather to visualize, document, and above all communicate effectively. To manage complexity it is advisable to organize diagrams by assigning them meaningful names, avoiding redundancy and using features such as packages or color-coding to group elements. Furthermore, it is important to be able to model a system using varying levels of abstraction depending on the specific development activity being carried out.

Slide 1.05.08 => Basic Use Cases

Use Cases and Actors: Use cases represent the specific goals

Escaneado con CamScanner

that users need to achieve capturing the system's visible functionality independently of design decision. These goals are pursued by "actors" entities external to the system, such as human users or other software or hardware systems. It is crucial to distinguish a use case from system interactions.

Diagrams and Relationships: Use case diagrams visually document these objectives and consist primarily of actors, use cases, relationships and subjects. Actors may have generalization or specialization relationships with one another. Use cases, meanwhile, are linked through three main types of relationships: <<extend>>, <<include>>, and generalization.

Benefits and Best Practices: Modeling use cases offers significant advantages: it helps in understanding who needs the system, uncovers opportunities to reuse components, and serves as a foundation for planning system development and testing. A well-structured diagram should focus on a single aspect of functionality and include only the essential elements. To improve readability, it is recommended to minimize crossing lines, group related elements together spatially, and use notes and colors as visual aids.

Slide 1.05.09 => Class Diagrams

Objects: These are the fundamental building blocks in object oriented systems, representing any "thing" that possesses boundaries, encapsulates state or behavior, and has its own distinct

Escaneado con CamScanner

identity. They generally do not represent events, pure behavior, or isolated constraints.

Classes: These are abstractions that allow for the grouping of entire collections of objects that share observable properties and characteristics. Class names should be singular nouns written in UpperCamelCase format, and the use of technical implementation jargon should be avoided.

Visual syntax: In UML, a class is represented by a box divided into compartments detailing its name, attributes, operations, and responsibilities. The names of instantiated objects include the instance name and the class name, separated by a colon ":" and are represented as underlined.

Modeling Perspectives: Class diagrams can be interpreted from three distinct approaches that determine their level of abstraction

- Analysis: Describes the sets of objects within the problem domain
- Specification: Describes the interfaces of logical representations and the obligation to provide services
- Implementation: Details abstract data types and software components in languages such as C++ or Java.

Internal Components

- Attributes: These are the properties that hold the values of objects. Their naming convention is lowerCamerCase, and their

syntax includes visibility (public '+', private '-', protected '#') name, multiplicity, type, and initial value.

- Operations: These represent the ways in which objects interact (analysis) or functional methods (implementation). They also follow the 'lower Camel Case' convention and specify parameters and return types.

Types of Structural Relationships

- Association: Structurally specifies that objects of one class are connected to those of another. Associations can be named and may specify a direction of navigation. N-ary associations connect more than two classes.
- Multiplicity: Restricts the number of objects that can be linked to another object within an association. It is indicated by simple numbers (e.g., 1), an asterisk (*) for an arbitrary number of instances or specific ranges such as 0..* or 1..3.
- Generalization: Represents a hierarchy either in terms of conceptual inheritance from an analysis perspective, or in terms of implementation inheritance.
- Aggregation and Composition: Aggregation is a special type of association indicating that one class is "part of" another; it is represented by an empty diamond. Composition is a stricter form of aggregation in which the survival of the part depends on the existence of the container object; it is represented by a solid diamond.

Escaneado con CamScanner

- Dependency: Represented by a dashed arrow, it indicates that one component "uses" another, such that if the target component changes, the dependent component might need to be modified (e.g., <<creates>> or <<instance of>>).

Class diagrams function much like paragraphs in a technical article: each one should focus on a specific topic while maintaining a consistent level of abstraction. Common design errors include adopting an inappropriate perspective, mixing levels of abstraction or focusing exclusively on structure while completely ignoring the system's behavior.

Slide 1 of 10 ⇒ UML - Quick Reference Guide

Use Cases and Classes

To capture interactions and static structure, the guide details two primary models. Use case diagrams represent actors and their objectives in relation to the system; the allow for the breakdown of complex goals through inclusion dependencies (<<include>>) and the modeling of exceptional situations via extensions (<<extend>>). Class diagrams, on the other hand, model sets of domain objects and their connections. These use associations to represent persistent links and define hierarchical relationships through generalization, or membership relationships through aggregation and composition.

Escaneado con CamScanner

Activity and Interaction

Process flow is modeled using activity diagrams, which employ initial action, and final nodes to map out the path of a task. These diagrams enable the handling of concurrent threads via fork and join nodes and include actions for sending or accepting asynchronous signals. Interaction diagrams (communication and sequence diagrams) are used to analyze how objects collaborate. Communication diagrams highlight the channels through which objects exchange messages, whereas sequence diagrams focus on the chronological order using "lifelines" detailing whether messages are synchronous or asynchronous, or if they result in the creation of a new object.

State Diagrams

Finally state diagrams are used to model the lifecycle of an individual object, describing its modes of operation or conditions over time. Objects move between states via transitions that can be conditioned by boolean rules and triggered by events, executing atomic, uninterruptible actions during the change. Furthermore, they allow for the definition of more complex behaviors through constructs like internal activities, entry transitions, and exit transitions.

Escaneado con CamScanner